

4. naloga: Integracija dinamičnih sistemov v času - meteorološka napoved

1 Uvod

Atmosfersko dinamiko lahko poenostavljeno zapišemo s sistemom nelinearnih PDE, ki opisujejo kinematiko tekočine, termodinamiko in fizikalne procese, kot so npr. prenos toplote s sevanjem, sprememba količine vodne pare in agregatnega stanja vode v atmosferi, spremembe količine aerosolov (trdih delcev) v ozračju, izmenjavo toplote, gibalne količine in snovi (voda, trdi delci) med ozračjem, tlemi in oceanom. Sedem osnovnih meteoroloških enačb na kratko opišemo spodaj.

Eulerjeve gibalne enačbe opišejo pospešek delca zraka v referenčnem sistemu, ki se vrti skupaj z Zemljo:

$$\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} + 2\boldsymbol{\Omega} \times \mathbf{v} = -\frac{1}{\rho} \nabla p + \mathbf{g} + \mathbf{F}_r, \quad (1)$$

kjer je $\mathbf{v} \equiv u\mathbf{i} + v\mathbf{j} + w\mathbf{k}$ 3D vetrovni vektor. Člen $-(1/\rho)\nabla p$ je sila gradienta tlaka, ki pospešuje delec zraka v smeri nižjega pritiska. Člen $2\boldsymbol{\Omega} \times \mathbf{v}$ predstavlja sistemski Coriolisov pospešek, $\mathbf{F}_r$ predstavlja pojemek toka ob stiku s podlago, $\mathbf{g}$ pa označuje težnostni pospešek (tj. vsoto gravitacijskega in centrifugalnega pospeška).

S kontinuitetno enačbo opišemo zakon o ohranitvi mase - lokalna sprememba gostote je enaka konvergenци masnega toka:

$$\frac{\partial \rho}{\partial t} = -\nabla \cdot (\rho \mathbf{v}). \quad (2)$$

Prvi zakon termodinamike za gibajočo tekočino v termodinamskem ravnovesju pravi, da je sprememba notranje energije delca zraka enaka vsoti dovedene toplote in dela, ki ga delec zraka opravi. Za idealen plin tedaj velja:

$$c_p \frac{dT}{dt} - \frac{1}{\rho} \frac{dp}{dt} = \frac{1}{m} \frac{dQ}{dt}, \quad (3)$$

kjer je c_p specifična toplota pri konstantnem pritisku in dQ/dt hitrost diabatnega ogrevanja (npr. zaradi kondenzacije vodne pare, sevalnega prenosa toplote ipd.). Temperatura, pritisk in gostota zraka pa so povezane še z enačbo stanja idealnega plina

$$p = \rho RT, \quad (4)$$

kjer je $R = R^*/M$ specifična plinska konstanta zraka, ki jo dobimo prek deljenja splošne plinske konstante R^* z molsko maso zraka M .

Ohranitev vlage opišemo s sledečo enačbo:

$$\frac{\partial \rho q^{(k)}}{\partial t} = -\nabla \cdot (\rho q^{(k)} \mathbf{v}) + P_q^{(k)}, \quad (5)$$

kjer k označuje različne faze vode v zraku (plinasto, tekočo, trdno), $q = m_q/m_{zrak}$ pa t.i. razmerje mešanosti. Tipično je vsaka od teh faz simulirana posebej. $P_q^{(k)}$ označuje vire in ponore (npr. evaporacija je vir vodne pare, kondenzacija pa njen ponor). Celotnemu sistemu enačb rečemo tudi *primitivne enačbe*, prvič pa jih je zapisal norveški fizik in meteorolog Vilhelm Bjerknes (1904).

Atraktor dinamike Zemljinega ozračja je kar njena klimatologija, ki je določena z geološkimi dejavniki (razporeditev kopnega in morja, orografija), astronomskimi dejavniki, npr. z oddaljenostjo Zemlje od Sonca (ta določa velikost energijskega toka Sončnega sevanja), spremembami eliptičnosti Zemljine tirnice okrog Sonca, nagnjenostjo Zemljine osi glede na ravnino ekliptike ter spremembo nagiba osi, precesijo osi ter seveda s sestavo atmosfere (ta se spreminja in posledično se spreminja tudi atraktor). Tudi trajektorije spremenljivk na Zemlji so ujete - npr. temperatura v troposferi ob teh pogojih nikoli ne more doseči 80°C ali -120°C , imamo pa znotraj teh klimatoloških meja "bogato" dinamiko. V tej nalogi bomo skušali nevronske mreže na reanalizah vremena naučiti poenostavljene atmosferske dinamike.

1.1 Geopotencial

V nalogi bomo veliko pozornosti posvetili geopotencialu, zato si tu oglejmo nekaj njegovih lastnosti. Navpična komponenta vetra w je na velikih prostorskih skalah običajno nekaj redov velikosti manjša od vodoravnih komponent u in v , zato v globalnih vremenskih modelih pogosto predpostavimo, da so navpične sile na delec zraka v ravnovesju. Če smo dovolj stran od tal, v vertikalni smeri enačbe (1) ostane

$$0 = -\frac{1}{\rho} \frac{\partial p}{\partial z} - g, \quad (6)$$

iz česar dobimo hidrostaticni približek

$$\frac{\partial p}{\partial z} = -\rho g. \quad (7)$$

Izračuni meteoroloških polj običajno ne potekajo na ploskvah fiksne oddaljenosti od tal, temveč pogosto na ploskvah konstantnega tlaka. Višino tlačne ploskve p nad nivojem gladine morja, kjer je zračni tlak p_s , dobimo iz hidrostaticnega približka (7) in enačbe stanja idealnega plina (4) kot

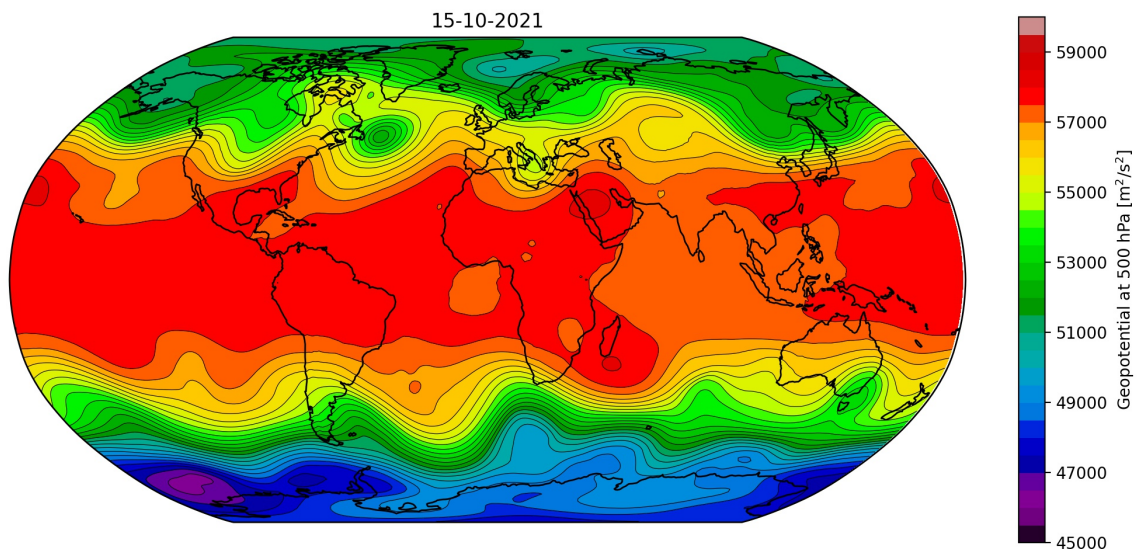
$$z(p) = - \int_{p_s}^p \frac{1}{\rho g} dp' = - \frac{R}{g} \int_{p_s}^p T(p') d \ln p'. \quad (8)$$

Geopotencial definiramo kot težnostno potencialno energijo na enoto mase:

$$Z(p) = gz(p) = -R \int_{p_s}^p T(p') d \ln p' = R\bar{T} \ln \frac{p_s}{p}. \quad (9)$$

Iz enačbe (9) prepoznamo, da višja povprečna temperatura stolpca zraka pod pritiskovo ploskvijo privede do večjega geopotenciala (zato je geopotencial nad tropi večji od geopotenciala nad poloma - glej sliko 1). V zmernih in visokih geografskih širinah lahko pri pojavih velikih prostorskih razsežnosti (npr. obsežni cikloni in anticikloni) iz geopotenciala v prvem približku odčitamo tudi smer in hitrost vetra. Ta namreč piha vzporedno z izolinijami geopotenciala, tako da ima na severni polobli nižji geopotencial na svoji levi, na južni pa na desni strani. Iz gradienta geopotenciala dobimo informacijo o hitrosti vetra, pri čemer večja gostota izoliniji pomeni višje hitrosti vetra. V globalnem polju geopotenciala lahko prepoznamo tudi t. i. Rossbyeve valove, ki meandrirajo polarno fronto in predstavljajo enega glavnih virov vremenske variabilnosti na sinoptičnih skalah (reda velikosti tisoč kilometrov) (glej sliko 1).

Geopotencial ima tudi velik zgodovinski pomen, saj je v prvi napovedi vremena (leta 1950) predstavljal edino prognostično spremenljivko [1]. V ta namen so izbrali 500 hPa tlačno ploskev, kjer je dobra izpolnjena predpostavka o barotropnosti tekočine (torej da je gostota le funkcija tlaka $\rho = \rho(p)$), kar pri nekaterih (tudi grobih) predpostavkah izrazito poenostavi dinamiko. Iz tega razloga je tudi v tej nalogi polje $Z500$ primarnega pomena.



Slika 1: Geopotencial na 500 hPa-ploskvi 15. oktobra 2021. Na ekvatorju je večji kot ob polih. V zmernih geografskih širinah so vidni Rossbyevi valovi. Približno na geografski širini Slovenije vidimo greben vala nad zahodno obalo ZDA, dolino v osrednjih ZDA in še en greben na vzhodni obali ZDA. Sledi dolina v zahodnem Atlantskem oceanu in še en greben ob evropski obali Atlantika...

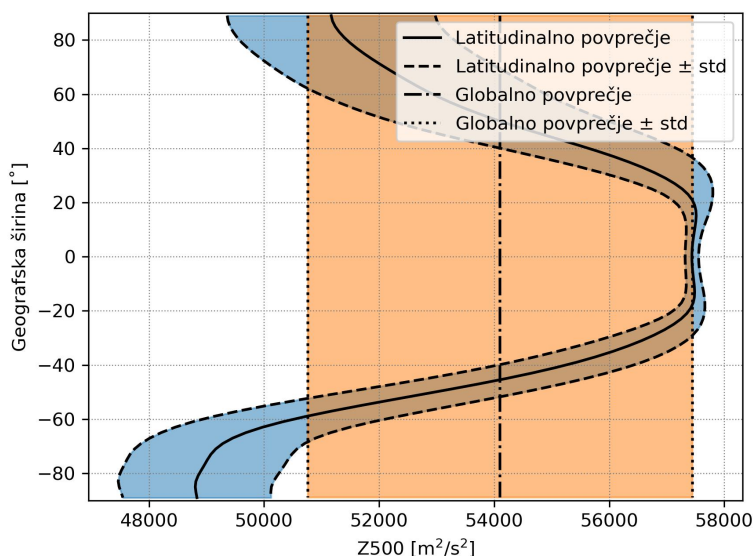
2 Podatki

Za učenje mreže uporabimo reanalize (rekonstrukcije vremena v preteklosti) ERA5 [2] Evropskega centra za srednjeročno napovedovanje vremena (ECMWF). Na voljo imamo dnevno povprečene podatke za geopotencial na 500 hPa ploskvi (Z500), temperaturo na višini 2m (T2m) in tlak pri tleh, preračunan na nivo gladine morja (MSLP), med leti 1940 in 2023. Podatki so na voljo na mreži z resolucijo 2° (90×180 točk), 4° (45×90) in 10° (18×36), kjer prvo število v oklepajih predstavlja število točk v smeri sever-jug, drugo pa v smeri vzhod-zahod. Poli so izvzeti iz podatkov, tako da so npr. pri 4° resoluciji v smeri sever-jug podatki zapisani za geografske širine od -88° do $+88^\circ$. Poleg meteorološki polj je na voljo tudi polje kopnega in morja (0=morje, 1=kopno), ki bi morda lahko nevronske mreže pomagalo z dodatnimi informacijami. Tega polja na izhodu ne napovedujemo.

Podatki po letih v formatu NetCDF (Network Common Data Form, glej Dodatek) so na voljo na Marvinu v mapi `/data/PSUF_naloge/4-naloga/era5`. Podatke standardiziramo za vsako spremenljivko posebej:

$$X \rightarrow \frac{X - \text{mean}(X)}{\text{std}(X)}, \quad (10)$$

kjer je $X \in \{Z500, T2m, MSLP\}$ in z `mean` in `std` označujemo klimatološko povprečje in standardni odklon (izračunana za obdobje 1981-2010). Pri implementaciji imate na voljo *'globalno'* standardizacijo, kjer Z500 povsod standardizirate z istima (globalno izračunanim) povprečjem in standardnim odklonom, ter *'latitudinalno'* standardizacijo, kjer pri vsaki geografski širini uporabite svoje standardizacijske vrednosti. Ker so značilnosti atmosfere močno odvisne od geografske širine (glej sliko 2), bi slednja morala olajšati delo nevronske mreže.



Slika 2: Globalno in latitudinalno klimatološko povprečje $\pm$ standardni odklon za Z500.

3 Metoda

V principu bi za napovedovanje lahko izbrali gosto povezano ali rekurentno nevronske mrežo. V praksi bi imeli težavo, da ena plast gosto povezanih nevronske mreže pri vhodnem in izhodnem vektorju velikosti $90 \times 180 = 16200$ nanese približno 260 milijonov učnih parametrov. To hito privede do preprileganja na učno množico in težave z računskimi kapacitetami. Tudi če to ne bi predstavljalo težave, imamo fizikalno predznanje o dinamiki atmosfere, da pri kratkoročnem napovedovanju na nadaljnji razvoj vremena v točki (λ, φ) , kjer je λ geografska dolžina in φ geografska širina, bolj vpliva stanje atmosfere v neposredni okolici, kot območje na drugi strani planeta. Večina povezav gosto povezanih nevronske mreže bi torej bila odveč. Zmanjšanje števila učnih parametrov in upoštevanje lokalnosti lahko dosežemo s konvolucijskimi nevronske mrežami.

3.1 Konvolucijske nevronske mreže (CNN)

Konvolucijske nevronske mreže posamezno komponentno izhodnega tenzorja z uporabo navzkrižne korelacije (ki ji zaradi podobnosti s konvolucijo rečemo kar konvolucija) povežejo s podmnožico vhodnega tenzorja. Iste uteži izbranega konvolucijskega jedra oz. filtra nato uporabijo tudi pri napovedi vseh ostalih komponent istega kanala izhodnega tenzorja, pri čemer na vhodu prejmejo druge podmnožice vhodnih podatkov. Na ta način se posamezno konvolucijsko jedro nauči lokalnih povezav med spremenljivkami, različna jedra pa so lahko namenjena detekciji različnih relevantnih vremenskih vzorcev oz. povezav. Z drugimi besedami bi lahko rekli, da se CNN učijo krajevne diskretizacije atmosferske dinamike, medtem ko v časovni dimenziji izvajajo direktno napovedovanje. Pri vsakem vhodnem tenzorju lahko nastavimo več filtrov, ki se učijo drugih povezav in vhodni tenzor preslikajo v svoj kanal.

V nalogi se bomo napovedovanja lotili s konvolucijskim kodirnikom (angl. convolutional encoder-decoder), kjer kodirni del vhodne podatke, v našem primeru meteorološko stanje $X(t)$, preslika v močno reduciran (*latentni*) prostor, v katerem nastane zgoščena reprezentacija informacij o vhodnem stanju. V dekodirniku se nato latentno stanje preslika v izhod, ki v naši

nalogi predstavlja meteorološko stanje ob času $X(t + \Delta t)$. Meteorološka polja so v konvolucijskem kodirniku oblike (B, H, W, C) , kjer je $B = \text{batch_size}$, $H = \text{višina}$, $W = \text{širina}$, $C = \text{število_kanalov}$. V vhodnem tenzorju vsak kanal predstavlja svojo meteorološko spremenljivko (vključno z masko kopno-morje). Vsaka izmed spremenljivk je tudi normalizirana, skladno z opisom v poglavju 2.

Primer CNN za eno samo meteorološko polje (tj. Z500, ki je predstavljen na mreži z resolucijo 4° (45×90 točk)) je zapisan v spodnji kodi:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import MaxPooling2D, UpSampling2D, Conv2D

def create_model_CNN():
    # Dimenzija vhoda naj bo (batch_size, nlat, nlon, st. polj)
    # Pri resoluciji 4 stopinje je nlat=45, nlon=90
    # Tu se k številu polj steje tudi maska kopnega in morja!
    # Za preverjanje teh 3 števil si naprintajte npr. np.shape(x_train)
    input_state = keras.Input(shape=(45,90,1))

    # ENCODER
    x = Conv2D(filters=2, kernel_size=(5, 5), activation='relu',
               padding="valid")(periodic_pad(input_state, (5,5)))

    x = MaxPooling2D((3, 3))(x)
    x = Conv2D(filters=4, kernel_size=(3, 3), activation='relu',
               padding="same")(x)

    x = MaxPooling2D((3, 3))(x)
    encoded = Conv2D(filters=8, kernel_size=(3, 3), activation='relu',
                     padding="same")(x)

    # DECODER
    x = UpSampling2D((3, 3))(encoded)
    x = layers.Conv2D(filters=4, kernel_size=(3, 3), activation='relu',
                     padding="same")(x)

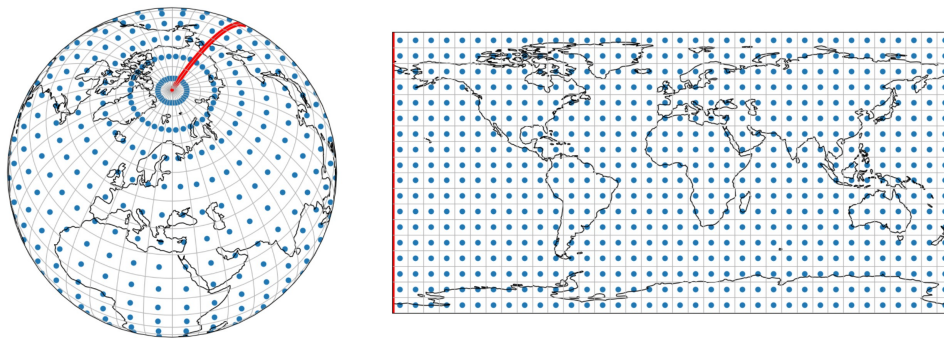
    x = UpSampling2D((3, 3))(x)
    # V zadnji plasti naj bo toliko filtrov, kot je izhodnih polj
    # (polja kopna in morja NE napovedujemo!)
    decoded = layers.Conv2D(filters=1, kernel_size=(3, 3), activation='linear',
                             padding="same")(x)

    encoder_decoder = keras.Model(input_state, decoded)
    print(encoder_decoder.summary())
    encoder_decoder.compile(optimizer=keras.optimizers.Adam(),
                            loss="mean_squared_error",
                            metrics=['mae', 'mse'])

    return encoder_decoder
```

Konvolucijske plasti delamo s `Conv2D`. Temu podamo število filtrov (konvolucijskih jeder, kar je enako številu izhodnih kanalov), dimenzijo konvolucijskih jeder, aktivacijsko funkcijo in *padding* (sl. *oblaganje*). Slednjega lahko nastavimo na 'same' ali 'valid', pri čemer 'valid' pusti vhodna polja v izvirnem stanju (zato imajo izhodna polja manjšo dimenzijo od vhodnih), 'same' pa vhodni tenzor v H in W dimeziji enakomerno oblega z dodatnimi vrednostmi. Če imamo nastavljeno obleganje 'same' in 'stride' 1 (velikosti korakov pri premikanju konvolucijskega jedra po vhodnih poljih) se na izhodu dimenzije plasti ne spremenijo. V primeru (3×3) jedra nam tako okrog našega polja doda po eno ničlo, v primeru (3×5) pa po eno ničlo na severu in jugu ter po dve ničli na vzhodu in zahodu.

V meteorologiji so polja sicer predstavljena na 2D mreži, vendar gre dejansko za polje na sferi, ki ga raztrgamo pri datumski meji in na polih (slika 3). Zato želimo z ustreznim obleganjem

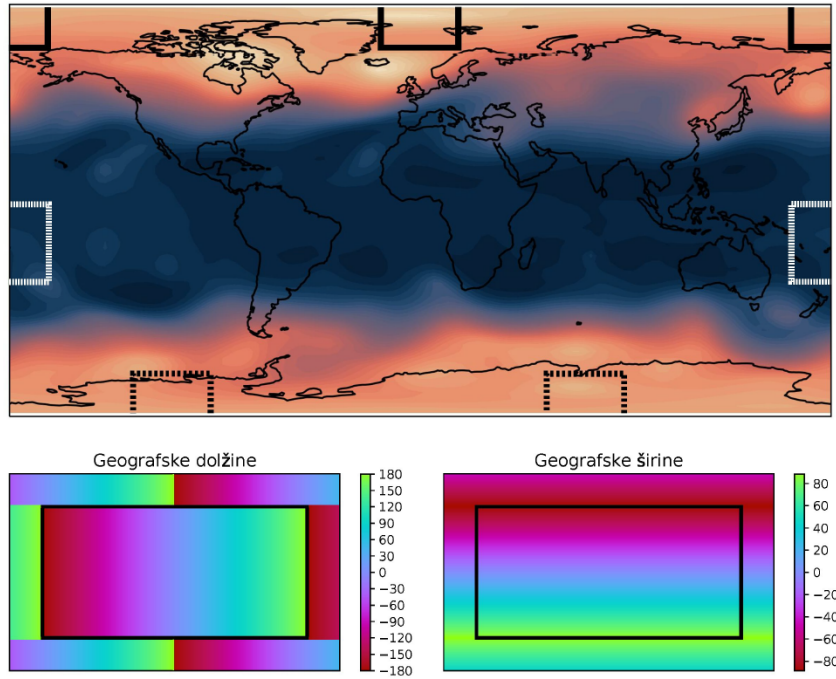


Slika 3: Mreža za predstavitev meteorološkega polja s fiksno resolucijo 10° v longitudinalni in latitudinalni smeri na sferi (levo) in kot 2D-polje v `numpy.array()` (desno). Središča celic so označena z modrimi pikami, robovi s sivimi črtami, datumsko meja je poudarjena z odebeljeno rdečo črto.

upoštevati periodični robni pogoj - glej sliko 4 (npr. $Z500(45^\circ, 179^\circ)$ (geog. širina, geog. dolžina) bo imel podobno povezavo z $Z500(45^\circ, 177^\circ)$ kot z $Z500(45^\circ, -179^\circ)$, vendar sta brez ustreznega obleganja (paddinga) daleč narazen – podobno velja za povezanost $Z500(89^\circ, 120^\circ)$ z $Z500(87^\circ, 120^\circ)$ in $Z500(89^\circ, -60^\circ)$). Ustrezen padding je v funkciji `create_model_CNN` uporabljen samo v prvi konvolucijski plasti, kjer kličemo funkcijo

```
def periodic_pad(input, kernel_size):
    """
    Implementira periodični robni pogoj na datumski meji in navzkrižnega
    na polih.
    kernel_size mora biti tuple dveh lihih števil (smer N-S, smer E-W)
    """
    NorthSouth_pad = kernel_size[0]//2
    EastWest_pad = kernel_size[1]//2

    # NORTH AND SOUTH PADDING
    if NorthSouth_pad > 0:
        # Top of matrix transformation
        top = tf.reverse(input[:,0:NorthSouth_pad,:,:], axis=(-3,))
        top = tf.roll(top, shift=int(top.shape[-2]/2), axis=-2)
        # Bottom of matrix transformation
        bottom = tf.reverse(input[:, -NorthSouth_pad:,:,:], axis=(-3,))
        bottom = tf.roll(bottom, shift=int(bottom.shape[-2]/2), axis=-2)
```

Slika 4: Območja, ki jih morajo pri projekciji iz sfere na ravnino pokrivati konvolucijska jedra na robu polja (zgoraj) in obleganje, s katerim to dosežemo (spodaj).

```
# Stack together
arr = tf.concat((top, input, bottom), axis=-3)
else:
    arr = input

# EAST AND WEST PADDING
if EastWest_pad == 0:
    return arr
else:
    left = arr[:, :, 0:EastWest_pad, :]
    right = arr[:, :, -EastWest_pad:, :]
    arr = tf.concat((right, arr, left), axis=-2)
    return arr

return pad_lon_lat
```

Ker so tudi v skritih plasteh sosednji nevroni tesno povezani, bi bilo fizikalno smiselno tak padding uporabiti v vsaki konvolucijski plasti.

V kodirniku manjšamo dimenzijo polja s *poolingom* (sl. združevanje) (npr. `MaxPooling2D((3,3))` bo zmanjšal dimenzijo za faktor 3 v obeh dimenzijah, tako da bo novo polje sestavljeno samo iz maksimalnih vrednosti 3×3 podmatrik starega polja), v dekodirniku pa počnemo obratni postopek z *upsamplingom* (sl. vzorčenje navzgor). V `create_model_CNN` z izjemo izhodne vsaki konvoluciji sledi bodisi pooling bodisi upsampling, vendar imamo v praksi lahko tudi več zaporednih konvolucij. V množici poolingov in upsamplingov se nam lahko hitro zgodi, da se uštejemo pri končni dimenziji (sploh če vhodno in izhodno polje ni tako prijetne dimenzije kot 45×90), zato je smiselno na koncu modela še printati ukaz `model.summary()`, ki nam našteje

vsako spremembo dimenzije v modelu in pove število učnih parametrov mreže v vsaki plasti (slika 5).

Model: "model"			
Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 45, 90, 1)]	0	
tf.__operators__.getitem_1 (Sli (None, 45, 2, 1))		0	input_1[0][0]
tf.__operators__.getitem_2 (Sli (None, 45, 2, 1))		0	input_1[0][0]
tf.compat.v1.shape (TFOpLambda) (4,)		0	input_1[0][0]
tf.concat (TFOpLambda)	(None, 45, 94, 1)	0	tf.__operators__.getitem_1[0][0] input_1[0][0] tf.__operators__.getitem_2[0][0]
tf.__operators__.getitem (Slici ())		0	tf.compat.v1.shape[0][0]
tf.__operators__.getitem_3 (Sli (None, 2, 94, 1))		0	tf.concat[0][0]
tf.compat.v1.floor_div (TFOpLam ())		0	tf.__operators__.getitem[0][0]
tf.__operators__.getitem_4 (Sli (None, 2, 94, 1))		0	tf.concat[0][0]
tf.roll (TFOpLambda)	(None, 2, 94, 1)	0	tf.__operators__.getitem_3[0][0] tf.compat.v1.floor_div[0][0]
tf.roll_1 (TFOpLambda)	(None, 2, 94, 1)	0	tf.__operators__.getitem_4[0][0] tf.compat.v1.floor_div[0][0]
tf.reverse (TFOpLambda)	(None, 2, 94, 1)	0	tf.roll[0][0]
tf.reverse_1 (TFOpLambda)	(None, 2, 94, 1)	0	tf.roll_1[0][0]
tf.concat_1 (TFOpLambda)	(None, 49, 94, 1)	0	tf.reverse[0][0] tf.concat[0][0] tf.reverse_1[0][0]
conv2d (Conv2D)	(None, 45, 90, 2)	52	tf.concat_1[0][0]
max_pooling2d (MaxPooling2D)	(None, 15, 30, 2)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 15, 30, 4)	76	max_pooling2d[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 5, 10, 4)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 5, 10, 8)	296	max_pooling2d_1[0][0]
up_sampling2d (UpSampling2D)	(None, 15, 30, 8)	0	conv2d_2[0][0]
conv2d_3 (Conv2D)	(None, 15, 30, 4)	292	up_sampling2d[0][0]
up_sampling2d_1 (UpSampling2D)	(None, 45, 90, 4)	0	conv2d_3[0][0]
conv2d_4 (Conv2D)	(None, 45, 90, 1)	37	up_sampling2d_1[0][0]
Total params: 753			
Trainable params: 753			
Non-trainable params: 0			
None			

Slika 5: `summary()` za `create_model_CNN`. Vse plasti, ki se začnejo s `tf.`, so posledica klicanja funkcije `periodic_pad`. Skupno število učnih parametrov tega modela je 753. Če bi želeli definirati mrežo z enakimi dimenzijami vmesnih polj, vendar pri tem zamenjati konvolucijske plasti z gosto povezanimi, bi ta imela 10^8 uteži.

Globok konvolucijski kodirnik lahko trpi izgubo ali eksplodiranje gradientov, kar zavira uspešno učenje. Temu se dodobra izognemo z dodajanjem preskočnih povezav (angl. *skip connections*), kjer kopiramo tenzorje kodirnika in jih v dimenziji kanalov konkatenujemo na tenzor enake oblike dekodirnika. V backpropagaciji tako močno skrajšamo verigo gradientov do izbrane plasti kodirnika. Dodatno lahko učenje izboljšamo z uporabo `x=BatchNormalization()(x)`, kjer vhomom v skrite plasti vzdržujemo gaussovske oblike, kar pohitri učenje (glej *covariate shift*).

3.2 Rekurentna nevronska mreža

Za iterativno napovedovanje bi lahko uporabili tudi mrežo LSTM, a bi vhodna polja imela zelo velike dimenzije (npr. $45 \cdot 90 = 4050$ točk na vsakem vhodu). Namesto tega bi lahko uporabili konvolucijski LSTM, ki gosto povezane plasti nadomesti s konvolucijskimi. V Kerasu je to implementirano kot `ConvLSTM2D`. Te možnosti v nalogi ni obvezno preizkusiti, bi pa rezultati lahko bili zanimivi.

4 Ugotavljanje uspešnosti natreniranih modelov

Osnovni meri za ocenjevanje uspešnosti meteoroloških modelov sta koren povprečnega kvadrata napake (RMSE) in korelacijski koeficient anomalij (ACC). Prvega izračunamo z

$$\text{RMSE} = \sqrt{(\mathbf{X}^f(t) - \mathbf{X}^t(t))^2}, \quad (11)$$

kjer je $\mathbf{X}^f$ napovedano polje, $\mathbf{X}^t$ privzeta „resnica“ (iz reanalize), povprečimo pa po vseh točkah mreže. Ker je v realnosti površina polja na sferi sorazmerna s kosinusom geografske širine (glej sliko 3), lahko prispevke kvadratov razlik pri geografski širini φ utežimo s $\cos(\varphi)/\overline{\cos(\varphi)}$, kjer velja $\overline{\cos(\varphi)} = (\sum_{j=1}^{N_\varphi} \cos(\varphi_j))/N_\varphi$. ACC dobimo kot

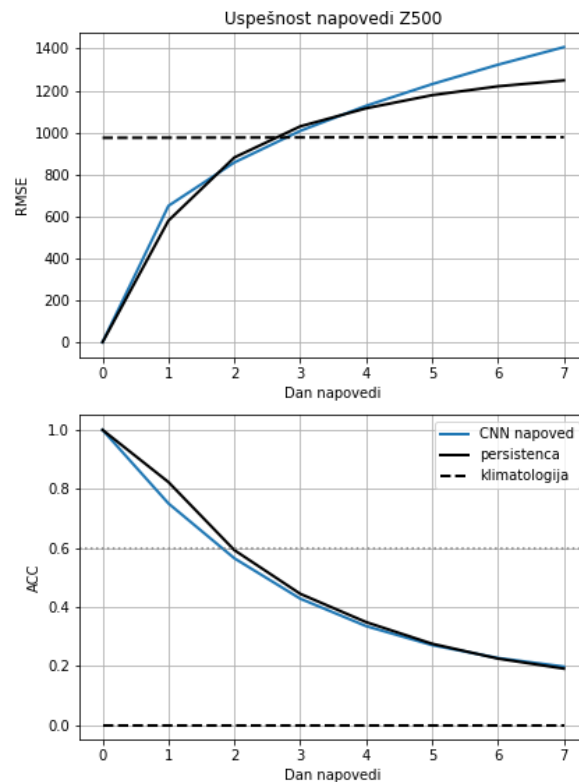
$$\text{ACC} = \frac{(\mathbf{X}^f(t) - \mathbf{X}^c(t)) \cdot (\mathbf{X}^t(t) - \mathbf{X}^c(t))}{\sqrt{(\mathbf{X}^f(t) - \mathbf{X}^c(t))^2} \sqrt{(\mathbf{X}^t(t) - \mathbf{X}^c(t))^2}}, \quad (12)$$

kjer je $\mathbf{X}^c(t)$ vektor klimatološkega stanja ob času t , torej stanje v nekem zaporednem dnevu v letu, povprečeno po 30-letnem obdobju (1981-2010). Tudi tu lahko površinsko utežimo prispevke h klimatološki kovarianci (v števcu) in klimatološkim variancam (v imenovalcu). Vrednost $\text{ACC} = 1$ pomeni popolno koreliranost, $\text{ACC} = 0$ pa popolno nekoreliranost napovedi in resnice. Subjektivna meja za uporabne napovedi na globalni ravni je $\text{ACC} > 0.6$.

RMSE in ACC modela lahko primerjamo s tema metrikama za *persistenčno* in *klimatološko napoved*. Persistenčno napoved definiramo kot napoved, kjer stanje atmosfere ostane enako začetnemu za vse čase napovedi. Klimatološko napoved pa definiramo kot napoved, kjer je atmosfera ob času napovedi enaka njenemu klimatološkemu povprečju (ob času napovedi). Napoved modela ob nekem času je informativna, če ima manjši RMSE in večji ACC od persistence in klimatologije. Primer vrednosti RMSE in ACC je na sliki 6.

5 Naloga

Cilj naloge je ustvariti in raziskovati nevronske model za napovedovanje vremena. Napoved bomo izvajali tako, da stanje $X(t)$ preslikamo v $X(t + \Delta t) = \mathcal{M}(X(t))$, pri čemer izhod iterativno uporabljamo kot vhod za naslednji časovni korak. Na spletni učilnici in na Marvinu (v direktoriji `"/data/PSUF_naloge/4-naloga"`) je na voljo Python skripta, ki jo kopiramo v svojo direktorijo (na Marvinu uporabimo ukaz `cp pot_do_python_skripte ciljna_direktorija`) in nastavimo potrebne poti. Za trening CNN uporabi podatke med leti 1979-2017, za validacijo pa 2018-2022. O uspešnosti modelov veliko pove že končna vrednost funkcije izgube, vendar ta opiše le kakovost 1. koraka napovedi pri izbrani metriki. Za boljšo sliko zato nariši tudi graf povprečne časovne odvisnosti RMSE in ACC za leto 2023, kot na sliki 6. Za kvalitativno primerjavo napovedanih polj lahko polja na sferi rišeš s knjižnico `cartopy` (glej Dodatek). Cilj je dobiti napoved, ki bo čim dlje uspešna. Najprej reši obvezno nalogo, nato pa podrobneje preuči



Slika 6: RMSE in ACC napovedi Z500 za podani primer CNN. Iz vidika RMSE je napoved uspešna samo 2. dan, iz vidika ACC pa nikoli. Oba pogoja torej nista nikoli hkrati izpolnjena, zato ta napoved ni nikoli celokupno uspešna.

katere izmed izbirnih nalog.

Obvezna naloga:

- Optimiziraj konvolucijski kodirnik za napovedovanje Z500 na prostorski ločljivosti 4° s časovnim korakom 1 dan. Kot osnovo vzemi mrežo iz navodil (oz. python skripte) in podatke z latitudinalno standardizacijo. Analiziraj odvisnost kvalitete mreže od:
 - števila filtrov,
 - dimenzije konvolucijskih jeder,
 - izbire aktivacijske funkcije
 - dodajanja ali odvzemanja konvolucijskih plasti z dodatnim poolingom in upsamplingom,
 - uporabe zero-paddinga ali periodičnega paddinga (namig: glej **kvalitativno** razliko po nekaj iteracijah napovedi),
 - (dodatno: *) uporabe polja morja in kopnega na vhodu v CNN,
 - (dodatno *) dodajanja plasti BatchNormalization
 - (dodatno **) dodajanja preskočnih povezav (angl. *skip connections*)
 - (dodatno **) dodajanja konvolucijskih plasti brez dodatnega poolinga in upsamplinga (to v praksi pomeni, da imaš npr. 2 zaporedni konvolucijski plasti brez vmesnega poolinga ali upsamplinga),

Dodatne naloge: (več * pomeni večjo okvirno zahtevnost naloge)

2. (*) Klasični numerični meteorološki modeli so iz numeričnih razlogov prisiljeni uporabljati iterativno napovedovanje z relativno kratkim časovnim korakom. Pri napovedovanju z nevronskimi mrežami pa teh omejitev načeloma ni. Kot pri Lorenzovem '63 modelu nauči model napovedovanja Z500 z daljšim časovnim korakom $X(t) \rightarrow X(t + n\Delta t)$.
3. (*) Kje na Zemlji je napaka napovedi največja? Kako se porazdelitev napak spremeni na testni množici v letu 1975.
4. (**) Kakovost rekonstrukcije vremena v reanalizah je odvisna od števila in kakovosti meritev, zato se z oddaljevanjem v zgodovino kot posledica tedajšnjega manj kakovostnega opazovalnega sistema slabša. Starejši podatki so torej slabši tudi za učenje modela. Poleg tega se spreminja tudi podnebje, zato lahko z učenjem modela na teh podatkih potencialno učimo model napačne klime. Ali se napovedna zmogljivost izboljša, če namesto direktnega učenja na celotni podatkovni množici ločeno učimo model najprej na starejših podatkih (npr. 1940 - 2000) in kasneje z naučenimi utežmi izvedemo "fine tuning" na novjših podatkih (npr. 2000 - 2018)? (validacijske in testne množice ne spreminjaj)?
5. (**) Kako na napoved vpliva prostorska resolucija vhodnega polja?
6. (**) Ali se napoved Z500 izboljša, če CNN hkrati napoveduje tudi T2m in/ali MSLP? Kako pa je s kvaliteto napovedi teh dveh spremenljivk?
7. (**) Kako na napoved vpliva normalizacija. Primerjaj napoved pri globalni in latitudinalni standardizaciji. Kako in zakaj se spremeni kakovost napovedi, če so standardizirane vrednosti ene spremenljivke (npr. Z500) 10 krat večje od ostalih spremenljivk (pazi, da pri destandardizaciji še vedno dobiš prave vrednosti).

Dodatek

Knjižnici netCDF in cartopy

Za odpiranje netcdf datotek uporabimo paket `netCDF4`. Če delamo z Anacondo, ga namestimo z ukazom

```
conda install -c anaconda netcdf4
```

Priložena koda za to nalogo je spisana tako, da je predprocesiranje podatkov opravljeno s funkcijo `dodeli_standardizirane_podatke()`, ki podatke vrne v obliki `numpy.array()`. Več o sami knjižnici `netCDF4` lahko preberete v dokumentaciji na <https://unidata.github.io/netcdf4-python/>.

Za risanje meteoroloških polj v različnih projekcijah potrebujemo paket `cartopy`. Z Anacondo ga namestimo z ukazom

```
conda install -c conda-forge cartopy
```

Za risanje pa lahko uporabite sledečo kodo, pri čemer lahko uporabite še množico drugih projekcij:

```
def plot_meteorological_field(meteorological_field, kolicina, naslov):  
    vmins = {'Z500':42000, 'T2m':190, 'MSLP':92000}  
    vmaxs = {'Z500':60000, 'T2m':320, 'MSLP':108000}
```

```

cmaps = {'Z500':'nipy_spectral', 'T2m':'terrain', 'MSLP':'bwr'}

# Geografske dolzine
lons = np.arange(-180 + resolucija/2, 180 - resolucija/2 + 1e-5,
                 step=resolucija)
# Geografske sirine
lats = np.arange(90 - resolucija/2, -90 + resolucija/2 - 1e-5,
                 step=-resolucija)

lt = lats
ln = lons
lns, lts = np.meshgrid(ln, lt)
fig = plt.figure(figsize=(10, 6))
ax = fig.add_subplot(1, 1, 1, projection=ccrs.Robinson())
# V primeru napake
# "AttributeError: 'GeoAxesSubplot' object has no attribute '_autoscaleXon'"
# odkomentiraj naslednji dve vrstici
# (napaka se lahko zgodi zaradi spora med verzijama matplotlib in cartopy)
#ax._autoscaleXon = False
#ax._autoscaleYon = False

filled_c = ax.contourf(lns, lts, meteorological_field,
                      np.linspace(vmins[kolicina], vmaxs[kolicina], 30),
                      cmap=cmaps[kolicina],
                      transform=ccrs.PlateCarree())
line_c = ax.contour(lns, lts, meteorological_field,
                    levels=filled_c.levels,
                    linewidths=0.4, colors=['black'],
                    transform=ccrs.PlateCarree())

ax.coastlines()
ax.gridlines()
ax.set_global()

ax.set_title(naslov)
fig.colorbar(filled_c, ax=ax, fraction=0.045)
fig.tight_layout()
plt.savefig('fig.png')
plt.show()

```

Literatura

- [1] Jules G Charney, Ragnar Fjørtoft, and J von Neumann. Numerical integration of the barotropic vorticity equation. *Tellus*, 2(4):237–254, 1950.
- [2] Hans Hersbach, Bill Bell, Paul Berrisford, Shoji Hirahara, Andras Horanyi, Joaquin Munoz Sabater, Julien Nicolas, Carole Peubey, Raluca Radu, Dinand Schepers, Adrian Simmons, Cornel Soci, Saleh Abdalla, Xavier Abellan, Gianpaolo Balsamo, Peter Bechtold, Gionata Biavati, Jean Bidlot, Massimo Bonavita, Giovanna Chiara, Per Dahlgren, Dick

Dee, Michail Diamantakis, Rossana Dragani, Johannes Flemming, Richard Forbes, Manuel Fuentes, Alan Geer, Leo Haimberger, Sean Healy, Robin J. Hogan, Elias Holm, Marta Janiskova, Sarah Keeley, Patrick Laloyaux, Philippe Lopez, Cristina Lupu, Gabor Radnoti, Patricia Rosnay, Iryna Rozum, Freja Vamborg, Sebastien Villaume, and Jean Noel Thepaut. The ERA5 Global Reanalysis. *Quarterly Journal of the Royal Meteorological Society*, page qj.3803, may 2020. ISSN 0035-9009. doi: 10.1002/qj.3803. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/qj.3803>.